Day 10: Linux Authentication Log Hunt

Building an Authentication Timeline with journalctl, grep, and tail

Field	Details
Intern Name	Amrita Pathak
Date	12 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	12 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Why This Task Matters 4
5. Building and Extracting the Authentication Timeline 4
6. Authentication Timeline 8
7. Findings 9
7.1 A missing auth.log was informative, not an obstacle 9
7.2 One sudo invocation, multiple wrong passwords, one failure line 9
7.3 Sudo does not lock the account, Windows does 9
7.4 Credential caching produced a legitimate, distinct log event with no password prompt at all 9
7.5 Two extraction methods, tail and grep, agreed on the same evidence 9
8. Conclusion 10

Executive Summary

This task built an authentication timeline on the Kali Linux lab VM using journalctl, grep, and tail, applying the same underlying authentication-hunting concept used on Windows Event Viewer in an earlier exercise, but through Linux's own logging tools. A short, deliberate sudo authentication sequence was generated, one rejected password attempt followed by a successful one, then a third, distinct event was identified while investigating the same window: a later sudo invocation that opened a new session without re-prompting for a password at all, using a cached credential.

Two adjustments to the original plan were made along the way, both driven by what the evidence on this specific machine actually showed rather than by what was assumed in advance. First, this Kali installation does not maintain a traditional /var/log/auth.log file, so grep and tail were applied directly to journalctl's piped output instead of to a flat file. Second, entering two wrong passwords in a row during a single sudo prompt cycle produced only one authentication failure line in the log, not two, since PAM logs one failure per invocation rather than per keystroke attempt, a real and useful distinction from how Windows records each failed attempt as its own separate event.

The resulting timeline, one failure, one fresh success, and one cached-credential session, was independently confirmed twice, once through tail and once through a targeted grep pattern, with both methods returning matching results. The central finding of this task is that Linux and Windows authentication failure genuinely behave differently at the mechanical level, not just cosmetically, which has direct consequences for how an analyst should interpret repeated failures on each platform.

1. Objective

Generate or identify authentication activity on the Kali Linux lab VM, then use journalctl and grep to extract successful and failed authentication events from the system journal, building a timeline of at least three distinct, evidenced timestamps.

2. Tools and Environment

Item	Detail
Endpoint used	Kali Linux VM (kali)
Account used	kali (default local user, sudo-capable)
Log source	systemd journal (journalctl); /var/log/auth.log not present on this install
Tools used	sudo, journalctl, grep, tail
Events selected	1 authentication failure, 1 successful authentication, 1 cached-credential session

3. Key Terms

- **PAM (Pluggable Authentication Modules)**: the underlying framework nearly all Linux authentication runs through, whether from a login prompt, SSH, or sudo. It is the Linux equivalent of what handles authentication behind the scenes on Windows, and it is what actually writes the log lines this task searches for.

- **systemd journal**: a modern, structured, binary log store used by systemd-based Linux distributions, queried with journalctl rather than opened as a text file. Not every Linux system uses it, older or minimal distributions may rely only on flat text logs instead.

- **journalctl identifier (-t)**: a filter that narrows journalctl's output to entries tagged with a specific program name, such as sudo, similar in purpose to filtering by Source in Windows Event Viewer.

- **Pipe (|)**: a shell operator that feeds the output of one command directly into another as input, rather than printing it to the screen. It is what allows small, single-purpose tools like journalctl, grep, and tail to be combined into one useful query.

- **Credential caching (sudo timestamp)**: sudo does not require a password on every single invocation. After a successful authentication, it remembers that success for a short window, typically 15 minutes, and allows further sudo commands during that window without re-prompting. Windows authentication events do not carry a directly equivalent caching concept.

4. Why This Task Matters

Generating and locating authentication events is a skill first built on Windows using Event Viewer. This task deliberately repeats that skill on Linux, not as busywork, but because a real SOC analyst cannot choose which operating system produced the alert in front of them. The benefits this specific repetition builds are worth naming directly:

- **Platform-independent thinking**: the concept being investigated, proving whether an authentication attempt succeeded or failed and when, is identical across operating systems. Only the tools and log format change. An analyst who understands the concept first can pick up any platform's specific tool quickly; one who only memorized Event Viewer clicks cannot.

- **Recognizing that failure counting is not universal**: Windows logs a separate 4625 event for every single wrong password submitted. Linux's PAM logged one authentication failure line for an entire multi-attempt sudo prompt cycle. Treating these two counting behaviors as equivalent would lead to miscounting a real incident's severity on whichever platform was misunderstood.

- **Recognizing that lockout behavior is not universal**: Windows can lock an account out entirely after too many failures. Sudo does not lock the account at all, it simply ends that one invocation after too many wrong attempts and allows a fresh try immediately after. An analyst assuming Windows-style lockout applies everywhere would misjudge how much real risk a string of Linux sudo failures represents.

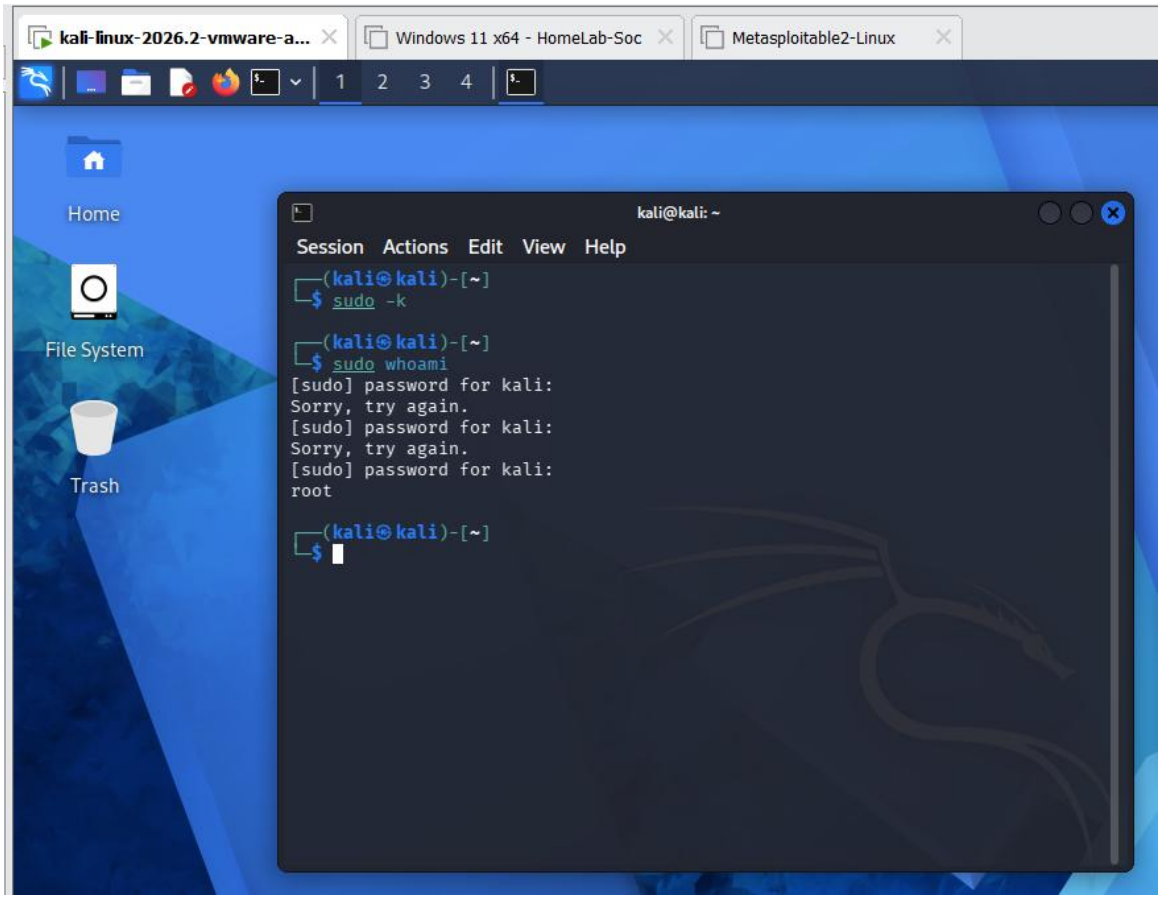
- **Comfort with adapting a plan to real evidence**: the original plan assumed a traditional auth.log file would exist. It did not on this machine. Adjusting the approach on the spot, rather than forcing the original plan to work, is a routine and necessary part of real investigative work, where no two environments are configured identically.

5. Building and Extracting the Authentication Timeline

This section documents the actual working process used to generate real authentication activity and then extract it using all three required tools, in the order the investigation was actually carried out. Every command actually run is shown highlighted, separate from the surrounding explanation.

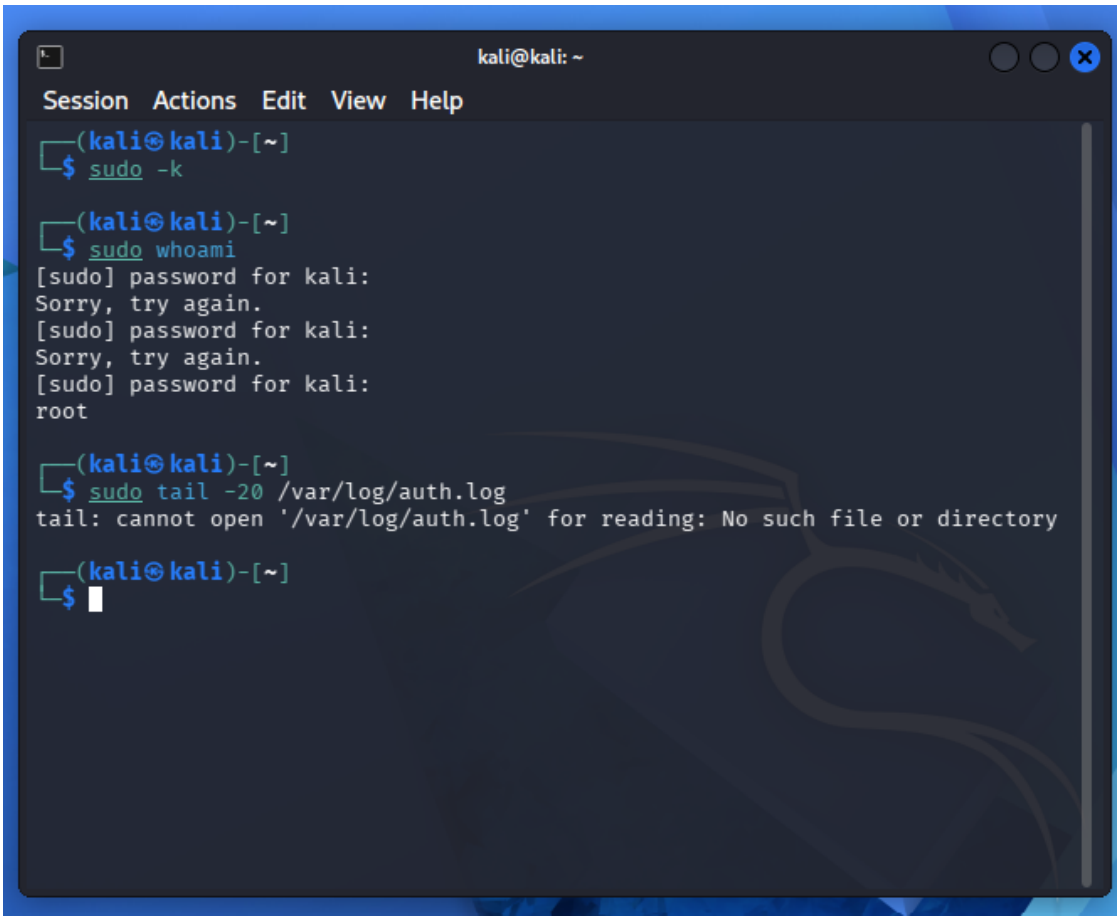
- **Step 1 - cleared cached sudo credentials:** ran `sudo -k` on the Kali terminal. This clears any cached authentication timestamp, guaranteeing the very next sudo command would genuinely prompt for a password rather than silently succeed from an earlier session.

- **Step 2 - generated one failed and one successful authentication attempt:** ran `sudo whoami` and deliberately entered the wrong password once, which sudo rejected, then entered the correct password on the immediate next prompt within that same invocation, which succeeded and correctly returned `root`.



Generating the authentication sequence: sudo -k, then sudo whoami with one rejected password followed by the correct one, confirmed by the returned value root.

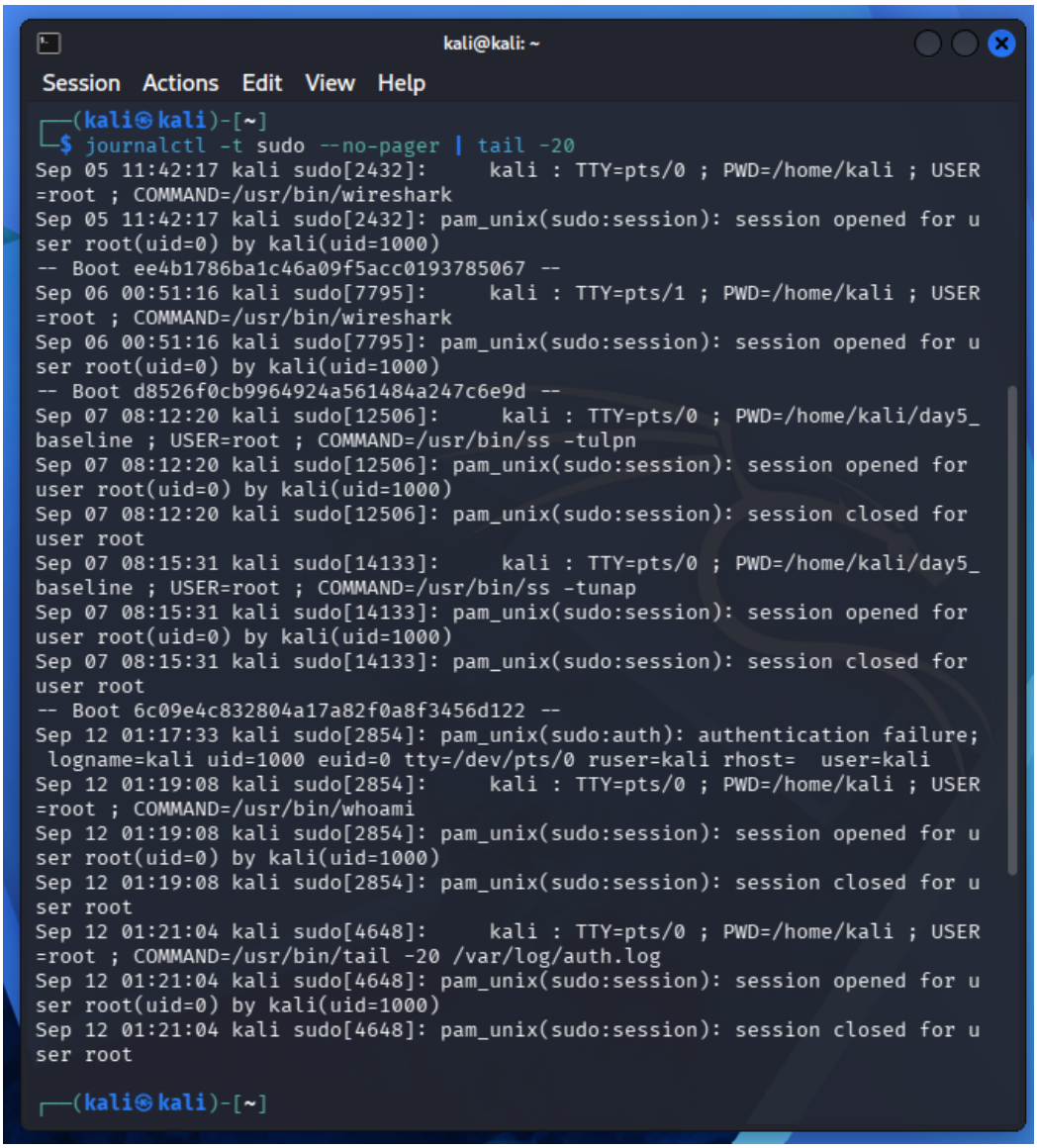
- **Step 3 - attempted to preview the flat log file with tail:** ran `sudo tail -20 /var/log/auth.log` to quickly check whether the attempt had been recorded. This is the standard first check on a traditional Linux system, but it returned an error instead of log lines.



tail reporting that /var/log/auth.log does not exist on this Kali installation.

This was not a dead end, it was itself a finding: this Kali installation does not maintain a traditional flat-file syslog for authentication, relying on the systemd journal exclusively. The plan was adjusted accordingly, using tail and grep on journalctl's piped output instead of on a file.

- **Step 4 - used journalctl and tail together to confirm the events were recorded:** ran `journalctl -t sudo --no-pager | tail -20`, filtering the journal to only sudo-tagged entries and then showing just the most recent twenty lines of that filtered output.



journalctl filtered to the sudo identifier, piped into tail, showing the generated sequence alongside older sudo activity already present on this machine.

This confirmed the generated attempt had been recorded, and also surfaced something worth noting honestly: this Kali VM already had real sudo activity going back to 4 September 2026, well before this exercise began. The timeline built in this report focuses on the sequence deliberately generated in Step 2 and confirmed in this step, since it can be fully accounted for command by command, but this machine's authentication history genuinely satisfies the task's use existing activity option as well, not only the generate new activity option.

- **Step 5 - used journalctl and grep together to isolate exactly the relevant lines:** ran `journalctl -t sudo --no-pager | grep -i "authentication failure\|session opened"`, searching the filtered journal output for lines containing either phrase, case-insensitively.



grep isolating only authentication failure and session opened lines from the sudo-filtered journal output, with matches highlighted.

The backslash before the pipe character inside the search pattern is required syntax, without it, grep would treat the pipe as a literal character to search for rather than as an or operator joining two patterns. This step proved the same events could be reliably isolated by pattern rather than only located by coincidence of appearing in the last 20 lines, which is the actual purpose of requiring grep as a separate tool from tail.

6. Authentication Timeline

Reading the confirmed output line by line surfaced one detail that changed how many distinct events this sequence actually produced. Two wrong passwords were entered, but both were rejected within a single sudo invocation, identifiable by the shared process ID sudo[2854] across every line of that attempt. PAM records one authentication failure line per invocation, not per password typed, so the two wrong entries appear as a single failure event, not two.

A separate later sudo command, run to preview the log itself, opened a new session under a different process ID, sudo[4648], without any password prompt at all, since it fell inside sudo's credential caching window. That gave three genuinely distinct, evidenced timestamps rather than the two originally expected.

Timestamp	Event	Process ID	Meaning
01:17:33	pam_unix(sudo:auth): authentication failure	sudo[2854]	Wrong password(s) rejected within one sudo invocation
01:19:08	pam_unix(sudo:session): session opened	sudo[2854]	Correct password accepted; fresh authentication succeeded
01:21:04	pam_unix(sudo:session): session opened	sudo[4648]	New sudo command run using cached credential; no password re-prompt

7. Findings

7.1 A missing auth.log was informative, not an obstacle

The absence of /var/log/auth.log on this machine was not a failure of the exercise, it accurately reflects how this specific Kali installation is configured, journal-only, no parallel flat-file syslog. Recognizing this quickly and adapting the grep and tail commands to operate on piped journalctl output instead, rather than assuming the file must exist somewhere else, is the same kind of environment-specific adjustment a real investigation constantly requires.

7.2 One sudo invocation, multiple wrong passwords, one failure line

Entering two wrong passwords produced exactly one authentication failure entry in the log, not two, because both attempts occurred within a single sudo invocation sharing one process ID. This is a meaningfully different counting behavior from the Windows Security log, where every individual wrong password submission generates its own separate 4625 event. An analyst counting Linux sudo failures by log line rather than by password attempt would undercount the true number of passwords tried within a single incident.

7.3 Sudo does not lock the account, Windows does

After the rejected attempt, the very next sudo prompt in the same invocation accepted the correct password immediately, with no lockout, delay, or warning of any kind. This stands in direct contrast to a typical Windows account lockout policy, where accumulating enough failed attempts would disable the account entirely for a period of time. This is a genuine platform difference with real investigative consequences: a long run of Linux sudo failures does not by itself indicate the account is now inaccessible to a legitimate user the way a Windows lockout would, so an analyst cannot apply Windows assumptions about account availability directly to a Linux host.

7.4 Credential caching produced a legitimate, distinct log event with no password prompt at all

The third timestamp in this timeline, a session opened at 01:21:04, involved no authentication prompt whatsoever, yet it is still a real, logged security-relevant event: a new elevated session was opened, using a credential established two minutes earlier. An analyst unaware that sudo caches credentials might mistake this line for an unexplained privilege escalation with no corresponding authentication, when in fact the authentication happened earlier and was simply still valid. Understanding this caching window is necessary to correctly attribute a cached-credential session back to its original authenticating event.

7.5 Two extraction methods, tail and grep, agreed on the same evidence

The events found through a plain tail preview and the events isolated through a targeted grep search were the same underlying log entries, retrieved two different ways. This kind of cross-method agreement is a useful evidentiary principle in its own right: using more than one method to arrive at the same result increases confidence that the evidence has not been misread or selectively filtered.

8. Conclusion

This task closes with a working timeline built entirely from evidence, not assumption: one rejected sudo attempt, one successful authentication, and one cached-credential session, extracted using sudo, journalctl, tail, and grep exactly as the task required.

What made this exercise worth doing was not the timeline itself, but how it was built. The plan changed twice along the way, once when the expected auth.log file was not present, and once when two wrong passwords turned out to log as a single failure rather than two, and both changes came from reading what the machine actually recorded rather than forcing the original assumption to fit. A further point worth carrying forward is that Linux's sudo does not lock an account after repeated failures the way Windows does, it simply ends that one attempt and allows a fresh try immediately, a real difference in how much risk a string of failed logins actually represents depending on the platform.